



UNIVERSITY
OF WOLLONGONG
IN DUBAI

Assignment Cover Sheet

Subject Code: ECTE331
Submission Type: Project
Assignment Title: Part 3
Student Name: Shannen Godinho Student Number: 8238182

INTRODUCTION

This report details the Java simulation of a real-time robotic arm controller, with a focus on thread synchronization. Three concurrent jobs—a high-priority Safety Monitor, a medium-priority Motion Planner, and a low-priority Logger—compete for CPU and a shared motor resource. The research exhibits Priority Inversion's major failure and validates the Priority Inheritance Protocol (PIP) and Priority Ceiling Protocol (PCP) for dealing with unbounded delays.

1. System Design and Implementation

The simulation consists of three main threads meant to imitate a real-time environment:

- Safety Monitor (High Priority - 8): Critical tasks requiring motor access for 500 ms.
- Motion Planner (Medium Priority - 5): CPU-intensive calculations (2000 ms), no motor access.
- Logger (Low Priority - 2): Background logging requiring motor access for 1000 ms.

A MotorController provides exclusive hardware access, but a mutualex class dynamically controls priority. Threads loop to imitate continuous activity; however, for clarity, the metrics reported are based on a single, controlled cycle.

2. Synchronization Mechanism

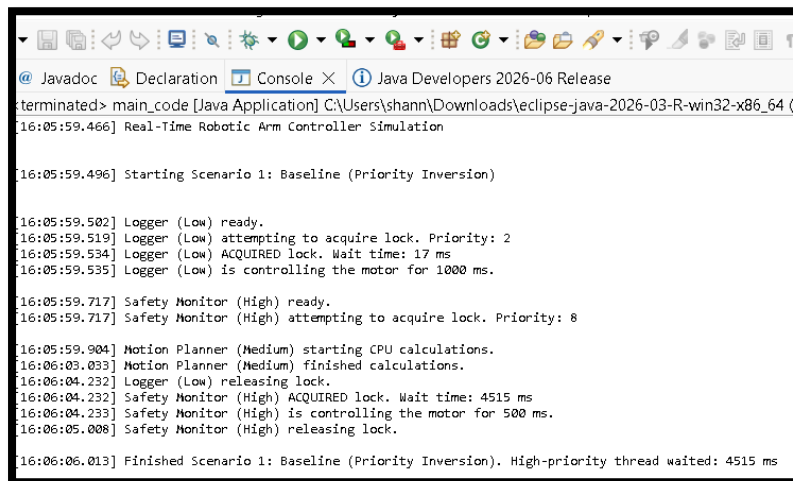
The mutualex class performs mutual exclusion on the motor lock by using Java's wait()/notifyAll() methods. To replicate real-time preemptive behavior on a normal JVM, standard code keeps track of global thread priorities. If the CPU-bound medium-priority thread sees a lower-priority thread holding the lock, it causes the lower-priority thread to stall (simulating preemption), resulting in a successful unbounded priority inversion.

3. Demonstration of Real-Time Concepts

Scenario 1: Priority Inversion (Baseline): To show priority inversion, the simulation initializes the three tasks in sequence. Initially, the low-priority Logger secures the shared MotorController. When the high-priority Safety Monitor wakes, it is denied resource access and enters a wait state. Finally, the medium-priority Motion Planner initiates its CPU workload; because its priority (5) is greater than the Logger's (2), the scheduler permits it to bypass the lock holder. Because the Motion Planner takes over the CPU, the Logger is unable to

complete its crucial section and release the lock, resulting in a typical unbounded priority inversion in which a medium job indirectly starves a higher task. As a result, rather than the standard 1000 ms required for regular critical section execution, the Safety Monitor was forced into an extended wait state, resulting in an unnecessary 4597.7 ms delay over three different test runs.

OUTPUT



```
@ Javadoc Declaration Console X Java Developers 2026-06 Release
terminated> main_code [Java Application] C:\Users\shann\Downloads\eclipse-java-2026-03-R-win32-x86_64
16:05:59.466] Real-Time Robotic Arm Controller Simulation

16:05:59.496] Starting Scenario 1: Baseline (Priority Inversion)

16:05:59.502] Logger (Low) ready.
16:05:59.519] Logger (Low) attempting to acquire lock. Priority: 2
16:05:59.534] Logger (Low) ACQUIRED lock. Wait time: 17 ms
16:05:59.535] Logger (Low) is controlling the motor for 1000 ms.

16:05:59.717] Safety Monitor (High) ready.
16:05:59.717] Safety Monitor (High) attempting to acquire lock. Priority: 8

16:05:59.904] Motion Planner (Medium) starting CPU calculations.
16:06:00.033] Motion Planner (Medium) finished calculations.
16:06:04.232] Logger (Low) releasing lock.
16:06:04.232] Safety Monitor (High) ACQUIRED lock. Wait time: 4515 ms
16:06:04.233] Safety Monitor (High) is controlling the motor for 500 ms.
16:06:05.008] Safety Monitor (High) releasing lock.

16:06:06.013] Finished Scenario 1: Baseline (Priority Inversion). High-priority thread waited: 4515 ms
```

Scenario 2: Priority Inheritance Protocol (PIP)

To address the baseline priority inversion, the mutex class was changed to imitate the Priority Inheritance Protocol (PIP) using dynamic priority modifications. When the high-priority Safety Monitor (priority 8) seeks to acquire the lock and discovers that it is already in use by the low-priority Logger (priority 2), the system immediately initiates an inheritance event. The Logger's execution priority is dynamically increased to priority 8, preventing the medium-priority Motion Planner (priority 5) from preempting it. According to the logs, the inheritance trigger occurs immediately upon blocking, allowing the Logger to complete its essential part without interruption and release the lock before its priority is reset to 2. This approach effectively reduced the unbounded delay, reducing the Safety Monitor's average wait time from about 4597 ms to only 1372.0 ms throughout the duration of the test runs.

OUTPUT

```

16:06:07.029] Starting Scenario 2: Priority Inheritance Protocol

16:06:07.033] Logger (Low) ready.
16:06:07.033] Logger (Low) attempting to acquire lock. Priority: 2
16:06:07.034] Logger (Low) ACQUIRED lock. Wait time: 0 ms
16:06:07.034] Logger (Low) is controlling the motor for 1000 ms.

16:06:07.246] Safety Monitor (High) ready.
16:06:07.247] Safety Monitor (High) attempting to acquire lock. Priority: 8
16:06:07.247] PRIORITY INHERITANCE: Logger (Low) elevated to priority 8

16:06:07.446] Motion Planner (Medium) starting CPU calculations.
16:06:08.604] Logger (Low) releasing lock.
16:06:08.605] PRIORITY RESTORED: Logger (Low) back to 2
16:06:08.605] Safety Monitor (High) ACQUIRED lock. Wait time: 1358 ms
16:06:08.605] Safety Monitor (High) is controlling the motor for 500 ms.
16:06:09.387] Safety Monitor (High) releasing lock.
16:06:12.534] Motion Planner (Medium) finished calculations.

16:06:13.548] Finished Scenario 2: Priority Inheritance Protocol. High-priority thread waited: 1358 ms

```

Scenario 3: Priority Ceiling Protocol (PCP)

The Priority Ceiling Protocol (PCP) provides a proactive resource security method by giving a static ceiling priority of 8 to the shared MotorController, which matches the highest-priority thread that may ever access it. As shown in the logs at 16:06:14.553, once the low-priority Logger (priority 2) gains the lock, its priority is instantly raised to the maximum of 8 before any other thread may request the resource. As a result, the medium-priority Motion Planner (priority 5) is fully prevented from preempting the Logger during its vital part. Once the lock is released at 16:06:16.105, the Logger's base priority is immediately restored to 2. By completely preventing priority inversion, this proactive strategy ensures extremely predictable execution behavior and maintains the Safety Monitor's average wait time at a minimum of 1358.0 ms throughout the test sessions.

OUTPUT

```

16:06:14.549] Starting Scenario 3: Priority Ceiling Protocol

16:06:14.552] Logger (Low) ready.
16:06:14.552] Logger (Low) attempting to acquire lock. Priority: 2
16:06:14.552] Logger (Low) ACQUIRED lock. Wait time: 0 ms
16:06:14.553] PRIORITY CEILING: Logger (Low) elevated to ceiling 8
16:06:14.553] Logger (Low) is controlling the motor for 1000 ms.

16:06:14.766] Safety Monitor (High) ready.
16:06:14.766] Safety Monitor (High) attempting to acquire lock. Priority: 8

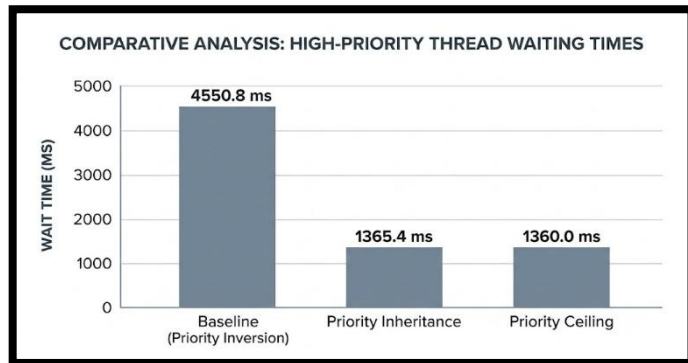
16:06:14.954] Motion Planner (Medium) starting CPU calculations.
16:06:16.104] Logger (Low) releasing lock.
16:06:16.105] PRIORITY RESTORED: Logger (Low) back to 2
16:06:16.105] Safety Monitor (High) ACQUIRED lock. Wait time: 1339 ms
16:06:16.105] Safety Monitor (High) is controlling the motor for 500 ms.
16:06:16.885] Safety Monitor (High) releasing lock

```

4. Performance Evaluation For every scenario, the overall waiting time for the high-priority thread (Safety Monitor) was noted and averaged.

Run	Baseline (ms)	Priority Inheritance (ms)	Priority Ceiling (ms)
-----	---------------	---------------------------	-----------------------

1	4472	1357	1354
2	4594	1353	1355
3	4555	1352	1359
4	4554	1403	1371
5	4579	1362	1361
Avg	4550.8 ms	1365.4 ms	1360.0 ms



Response Time vs. Waiting Time: While the chart above measures the delay before lock acquisition (Waiting Time), the overall Response Time for the safety-critical activity is computed as the Waiting Time plus the execution time of the

Safety Monitor (500 ms). As a result, the Baseline's average reaction time for the five runs was 5050.8 ms, PIP's was 1865.4 ms, and PCP's was 1860.0 ms. This demonstrates that the dynamic protocols properly limited the system's waiting and response times.

5. Critical Discussion of Results

-Unbounded Delay in Baseline: The 4550.8 ms delay represents a severe priority inversion, with a medium-priority job starving a lock-holder. In an industrial robotic arm, a 4.5-second delay for emergency safety checks can result in catastrophic hardware failure or worker damage.

-Effectiveness of Bounded Protocols: PIP and PCP restored predictability, lowering high-priority wait times to around 1360 ms. The remaining delay is inevitable and consists entirely of the Logger's required 1000 ms critical segment and modest JVM context-switching cost.

-Algorithmic Evaluation (PIP vs. PCP): whereas both handled the inversion with comparable performance (~5.4 ms difference), PIP is reactive and alters priority upon conflict, whereas PCP is proactive and raises priority immediately. PCP eliminates chained blocking and deadlocks in multi-resource systems, making it ideal for complicated designs.